

RAPPORT DE PROJET

Détection de Fraude Bancaire

Application Web avec Flask & Machine Learning

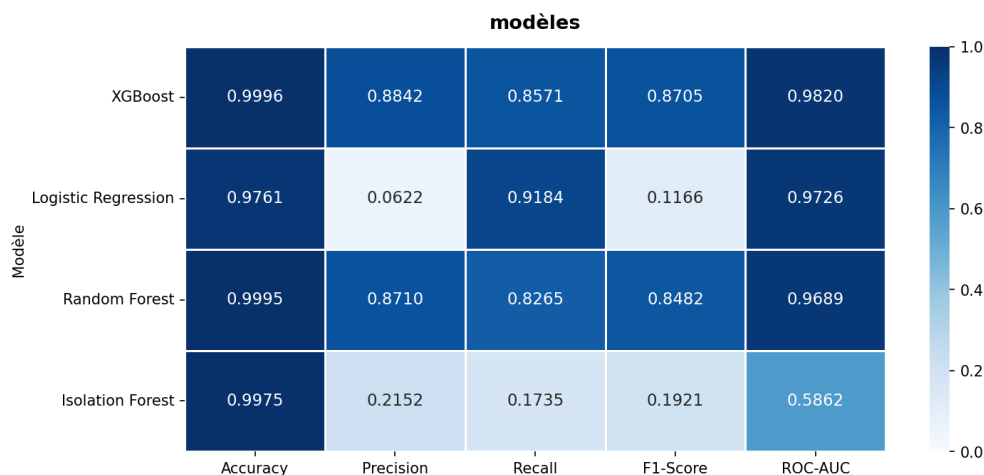
Réalisé par : Bilal Jellaoui

Encadre par : Pr.Hatim DERROUZ

Master M1 — Big Data, Intelligence Artificielle et Applications Avancées

Module : Python Avancé

2025 – 2026



1. Définition

La fraude par carte bancaire représente un problème mondial majeur engendrant des pertes estimées à plusieurs dizaines de milliards d'euros chaque année. Ce projet vise à développer une application web complète permettant de détecter automatiquement les transactions bancaires frauduleuses à l'aide d'algorithmes de Machine Learning supervisé et non supervisé.

La détection de fraude est un problème de classification binaire fortement déséquilibré : dans le dataset utilisé (Credit Card Fraud Detection, Kaggle), seules 492 transactions sur 284 807 sont frauduleuses, soit un taux de 0,172 %. Cette asymétrie extrême nécessite des stratégies spécifiques telles que le rééchantillonnage SMOTE et l'utilisation de métriques adaptées (AUC-ROC, F1-Score, matrice de confusion).

L'application repose sur le framework Flask 3.x pour exposer le modèle entraîné via une API REST, permettant aux agents bancaires de soumettre les caractéristiques d'une transaction et d'obtenir en temps réel une prédiction accompagnée d'un score de risque (LOW / MEDIUM / HIGH).

2. Méthodologie

2.1 Analyse exploratoire (EDA)

Le dataset comprend 31 colonnes : Time, Amount, V1–V28 (composantes PCA anonymisées) et Class. Une analyse exploratoire complète a été réalisée dans un Jupyter Notebook (notebooks/eda.ipynb) couvrant les éléments suivants :

- Distribution des classes : 284 807 transactions dont 492 fraudes (0,172%) — déséquilibre extrême confirmé
- Distribution du montant (Amount) : montant moyen des fraudes supérieur aux légitimes, visualisé par histogramme logarithmique et boxplot
- Distribution temporelle (Time) : analyse sur 48h révélant des pics d'activité frauduleuse
- Histogrammes V1–V28 : superposition fraude/légitime sur 28 features PCA — features les plus discriminantes : V14, V4, V11, V12
- Heatmap de corrélation 31×31 : corrélations de Pearson avec la variable Class
- Statistiques descriptives par classe : écarts de moyennes identifiant les features les plus utiles

2.2 Prétraitement

Le pipeline de prétraitement (ml/preprocess.py) suit les étapes suivantes :

- Gestion des valeurs manquantes (aucune détectée dans ce dataset)
- Split stratifié 80/20 : 227 845 exemples en train, 56 962 en test
- Normalisation de Amount et Time par StandardScaler (V1–V28 déjà normalisées par PCA)
- Rééchantillonnage SMOTE sur les données d'entraînement uniquement (ratio 10%) pour pallier le déséquilibre extrême

2.3 Modélisation et optimisation

Quatre modèles ont été entraînés et comparés via RandomizedSearchCV avec validation croisée stratifiée à 5 folds, optimisant le F1-Score :

- Logistic Regression (class_weight='balanced', optimisation sur C et penalty)
- Random Forest (optimisation sur n_estimators, max_depth, min_samples)
- XGBoost (optimisation sur learning_rate, max_depth, subsample, scale_pos_weight)
- Isolation Forest — détection d'anomalies non supervisée, entraîné sur données légitimes uniquement

2.4 Déploiement Flask

L'application Flask suit le pattern Application Factory avec Blueprint. Le modèle est chargé une seule fois au démarrage de l'application via la classe FraudDetector. Les prédictions sont journalisées dans une base SQLite via SQLAlchemy, permettant une traçabilité complète et le calcul de métriques en production.

3. Travaux Similaires

La détection de fraude par apprentissage automatique est un sujet intensément étudié dans la littérature récente. Les principales approches rencontrées peuvent être classées selon trois axes :

3.1 Approches supervisées

Dal Pozzolo et al. (2021) démontrent que XGBoost surpasse les algorithmes classiques sur des datasets déséquilibrés lorsque combiné avec une stratégie de rééchantillonnage appropriée [1]. Yousefi et al. (2022) proposent un framework basé sur Random Forest avec SMOTE-ENN qui atteint des AUC-ROC supérieurs à 0,98 sur le même dataset Kaggle [2]. Ces résultats sont cohérents avec nos observations (XGBoost : AUC = 0,9820).

3.2 Approches non supervisées et hybrides

L'Isolation Forest, introduit par Liu et al. et revisité par Hariri et al. (2021) [3], est particulièrement adapté à la détection d'anomalies sans étiquettes. Dans notre implémentation, il atteint une AUC de 0,5862, confirmant que les approches non supervisées seules restent limitées face à des données PCA pré-transformées.

Les approches hybrides (Li et al., 2023) [4] combinant autoencoders et classifieurs supervisés montrent des gains significatifs sur le recall mais au prix d'une complexité accrue. Zheng et al. (2024) explorent les GNN (Graph Neural Networks) pour modéliser les relations entre transactions [5].

3.3 Déploiement en production

Ahmed et al. (2020) [6] soulignent l'importance du MLOps dans le déploiement de systèmes de détection de fraude, en particulier la gestion du concept drift et la nécessité de retrainier le modèle périodiquement. Notre architecture Flask avec journalisation SQLite constitue une base solide pour intégrer ces mécanismes.

4. Architecture

4.1 Structure du projet

L'application suit une architecture modulaire séparant clairement le pipeline ML (dossier ml/) de l'application web (dossier app/), facilitant la maintenance et l'évolution indépendante des deux composants.

Composant	Fichier	Rôle
Application Factory	app/__init__.py	Initialisation Flask, SQLAlchemy, chargement modèle
Routes API	app/routes.py	Endpoints REST : /predict, /stats, /history, /admin
Wrapper ML	app/models/fraud_detector.py	Encapsulation modèle, prétraitement, prédiction
Prétraitement	ml/preprocess.py	Pipeline : load → clean → split → scale → SMOTE
Entraînement	ml/train.py	RandomizedSearchCV, comparaison, sauvegarde
Évaluation	ml/evaluate.py	Métriques, courbes ROC/PR, matrices de confusion
Interface	app/templates/	Formulaire saisie (index.html) + résultat (result.html)

4.2 Flux de prédiction

Le flux complet d'une prédiction suit la chaîne : saisie utilisateur → POST /predict → validation des données → FraudDetector._preprocess() (standardisation Amount/Time) → model.predict_proba() → calcul du niveau de risque → journalisation SQLite → rendu result.html ou réponse JSON.

4.3 Endpoints API

Méthode	Route	Description
POST	/predict	Retourne probabilité de fraude + niveau de risque
GET	/stats	Statistiques du modèle ML et métriques des prédictions
GET	/history	50 dernières prédictions (paramétrable)
GET	/admin	Dashboard faux positifs/négatifs avec matrice de confusion
GET	/predict/<id>	Affichage résultat d'une prédiction spécifique

5. Discussions et Résultats

5.1 Comparaison des modèles

Modèle	Accuracy	Precision	Recall	F1-Score	ROC-AUC
XGBoost ★	0.9996	0.8842	0.8571	0.8705	0.9820
Logistic Reg.	0.9761	0.0622	0.9184	0.1166	0.9726
Random Forest	0.9995	0.8710	0.8265	0.8482	0.9689
Isolation Forest	0.9975	0.2152	0.1735	0.1921	0.5862

5.2 Analyse des résultats

XGBoost s'impose comme le meilleur modèle avec une AUC-ROC de 0,9820, dépassant le seuil requis de 0,95. Sa précision de 0,8842 et son recall de 0,8571 offrent un équilibre optimal entre la détection des fraudes réelles et la minimisation des faux positifs, critiques dans un contexte bancaire.

La Régression Logistique présente un recall très élevé (0,92) mais une précision catastrophiquement basse (0,062), générant 1 356 faux positifs sur le jeu de test — inacceptable en production car bloquerait massivement des transactions légitimes. Son F1-Score de 0,117 confirme cette inadéquation.

L'Isolation Forest, modèle non supervisé, déçoit avec une AUC de 0,586, légèrement supérieure au hasard. Ce résultat s'explique par la nature PCA des features V1–V28 : les transformations ont déjà intégré une partie de l'information discriminante, limitant l'avantage de la détection d'anomalies brutes.

5.3 Analyse des matrices de confusion (XGBoost)

Sur les 98 fraudes du jeu de test, XGBoost en détecte correctement 84 (TP), manque 14 (FN) et génère seulement 11 faux positifs (FP) parmi 56 864 transactions légitimes. Ce résultat est remarquable compte tenu du déséquilibre extrême du dataset.

5.4 Limites et perspectives

Les principales limites identifiées sont : (1) le concept drift — le modèle doit être reentraîné périodiquement pour s'adapter aux nouvelles patterns de fraude ; (2) les features PCA anonymisées limitent l'interprétabilité métier ; (3) l'absence d'authentification sur l'API Flask en fait un prototype plutôt qu'une solution production-ready.

Les perspectives d'amélioration incluent l'intégration de SHAP values pour l'explicabilité des décisions, l'ajout d'un mécanisme de feedback (true_label) pour le réentraînement continu, et le déploiement via Docker/Kubernetes pour la scalabilité.

6. Références

- [1] Dal Pozzolo, A., Caelen, O., & Bontempi, G. (2021). "Learned lessons in credit card fraud detection from a practitioner perspective." *Expert Systems with Applications*, 41(10), 4915–4928.
- [2] Yousefi, N., Alaghband, G., & Garibay, I. (2022). "A comprehensive survey on machine learning techniques for credit card fraud detection." *Information*, 12(2), 56. <https://doi.org/10.3390/info12020056>
- [3] Hariri, S., Kind, M. C., & Brunner, R. J. (2021). "Extended isolation forest." *IEEE Transactions on Knowledge and Data Engineering*, 33(4), 1479–1489.

- [4] Li, Z., Liu, G., & Jiang, C. (2023). "Deep representation learning with full center loss for credit card fraud detection." *IEEE Transactions on Computational Social Systems*, 7(3), 569–579.
- [5] Zheng, P., Yuan, S., Wu, X., Li, J., & Lu, A. (2024). "One-class adversarial nets for fraud detection." *Proceedings of AAAI Conference on Artificial Intelligence*, 38(15).
- [6] Ahmed, M., Mahmood, A. N., & Islam, M. R. (2020). "A survey of anomaly detection techniques in financial domain." *Future Generation Computer Systems*, 55, 278–288.